

AtlasIT: Controlled MVP → Scalable Platform

5-page executive technical brief (PDF-ready)

Author: Joe (AtlasIT) • Date: 2026-03-14 (America/Chicago)

Executive summary

AtlasIT can reach a controlled MVP quickly **without incurring a rewrite at scale** by enforcing a strict separation between a shared **control plane**, tenant-scoped **execution plane**, and immutable **evidence plane**, starting Cloudflare-first for velocity and migrating to AWS once product/tenant fit is proven. The controlled MVP should prove one tight loop—**HRIS/Directory signal → JML workflow execution → connector actions → policy decisions → immutable evidence → console visibility**—with hard gating around durability, auth, contract parity, and evidence immutability. Cloudflare provides the right primitives for this MVP (Workers + D1 bindings + R2 bindings + Durable Objects alarms + Queues DLQ + Workers for Platforms dispatch namespaces), while long-lived posture at 100k tenants maps cleanly to AWS (Step Functions + DynamoDB + S3 with Object Lock) if AtlasIT builds to internal interfaces and keeps evidence as an append-only ledger. Cloudflare's platform binding model for D1/R2 and stateful coordination via Durable Objects alarms (with retries) supports the MVP execution architecture. ¹

Recommended repo file layout (minimal, migration-safe):

```
/services
  control-plane-api/      # tenants, onboarding, mappings, policy
assignments
  workflow-engine/        # DO + queues orchestration, step state, retries,
DLQ
  connector-runtime/      # adapter invocation, rate-limit handling,
idempotency
  evidence-writer/        # envelopes, hashing, storage write + index update
  policy-engine/          # OPA bundle fetch/verify, decision logging
  ai-governance/          # analyzers → proposals → gated actions
/packages
  contracts/              # OpenAPI + event schemas + evidence schema
  platform-abstractions/  # QueueBus, EvidenceStore, SecretResolver, etc.
  jml-templates/          # joiner/mover/leaver templates (YAML)
  observability/          # logging, trace ids, metrics events, SLO queries
/policies
  bundles/               # signed OPA bundles (build artifacts)
  rego/                  # source rego (built into bundles)
```

Platform architecture

Cloudflare-first architecture (controlled MVP)

```
flowchart LR
    A[Admin / Design Partner] --> C[Console]
    C --> CP[Control Plane API Worker]

    CP -->|Create run| WE[Workflow Engine<br/>Durable Object]
    CP -->|Read/write config| DB[(D1: tenant config + indexes)]
    CP -->|Artifacts| OBJ[(R2: immutable evidence + snapshots)]

    WE --> Q[(Queues: step tasks)]
    Q --> EX[Execution Workers]
    EX --> DISP[Dispatch Worker]
    DISP --> NS[Dispatch Namespace<br/>Adapter Workers]
    EX --> PE[Policy Engine<br/>OPA bundle/wasm]
    EX --> EW[Evidence Writer]

    PE -->|Decision logs| EW
    EW --> DB
    EW --> OBJ

    subgraph Observability
        LOG[Workers Logs]:::o
        AE[Analytics Engine dataset]:::o
    end
    end
    CP --> LOG
    EX --> LOG
    EW --> AE

    classDef o fill:#f5f5f5,stroke:#999,stroke-width:1px;
```

Why these Cloudflare primitives map well to MVP constraints: - **D1** binding provides a Worker-accessible relational store via `env.<binding>` for tenant config and evidence indexes. ²

- **R2** binding provides object storage accessible directly from Workers without separate SDK/key management in the request path; evidence artifacts can live in immutable object keys per run/step. ³

- **Durable Objects alarms** allow workflow runs to be resumed and retried in a fine-grained way, with at-least-once execution and automatic retries on failure—useful for step orchestration and “wake-ups” without relying on external traffic. ⁴

- **Queues** provide asynchronous fan-out (execute steps off-request), built-in retry configuration, and **dead letter queues** for nonrecoverable step failures—directly supporting workflow DLQ semantics. ⁵

- **Workers for Platforms dispatch namespaces** allow a single dynamic dispatch Worker to invoke any adapter Worker via `env.NAMESPACE.get("worker-name")`, with default isolation guarantees for user Workers. ⁶

- **Cloudflare Access** can front the console/API; the application should validate the `Cf-Access-Jwt-`

Assertion header using published signing keys and an audience tag. ⁷

- **Workers Logs** support structured logging and configurable head-based sampling to control volume at scale. ⁸

- **Analytics Engine** can store high-cardinality datapoints for per-tenant SLOs and cost attribution, queryable via SQL API. ⁹

Minimal portability interfaces and storage mapping

Platform abstractions (keep business logic stable across Cloudflare → AWS):

```
export interface QueueBus {
  publish(queue: string, msg: unknown, opts?: { delaySec?: number }):
  Promise<void>;
}

export interface WorkflowStateStore {
  getRun(runId: string): Promise<unknown | null>;
  putRun(runId: string, state: unknown): Promise<void>;
}

export interface EvidenceStore {
  putObject(path: string, body: Uint8Array, meta: Record<string, string>):
  Promise<{ uri: string }>;
}

export interface PolicyEvaluator {
  evaluate(input: unknown): Promise<{ decision: unknown; decisionId: string;
  bundleRevision: string }>;
}

export interface SecretResolver {
  resolve(secretRef: string): Promise<Uint8Array>; // never return plaintext to
  logs
}

export interface ConnectorInvoker {
  invoke(workerName: string, req: Request): Promise<Response>;
}
```

Cloudflare-first → AWS migration mapping (not a commitment; a clean target): | Logical subsystem |
Cloudflare MVP | AWS scale path | |---|---|---| | API ingress | Workers | API Gateway + Lambda / ECS | |
Workflow orchestration | Durable Objects + Alarms ⁴ | Step Functions Standard Workflows (durable,
long-running, auditable) ¹⁰ | | Async execution & DLQ | Queues + DLQ ¹¹ | SQS + DLQ / EventBridge | |
Config & indexes | D1 (tenant config + evidence index) ¹² | DynamoDB (high-cardinality partition keys)
¹³ | | Evidence artifacts | R2 ¹⁴ | S3 + Object Lock (immutability) ¹⁵ | | Adapter isolation | Dispatch
namespaces ¹⁶ | Dedicated per-adapter services / Lambda per connector | | Policy engine updates | OPA

bundles + decision logs ¹⁷ | OPA sidecar/bundle service + centralized logging | | CI/CD secrets | Vault JWT auth via GitHub OIDC ¹⁸ | Same (Vault) or Secrets Manager |

Phased roadmap with dates

Assumption: This is a proposed schedule starting the next business week. If internal staffing differs, dates are **unspecified** and should be recalculated.

```
gantt
  title AtlasIT Controlled MVP → Pilot Readiness (Proposed)
  dateFormat YYYY-MM-DD
  axisFormat %b %d

  section Hardening foundations
  Workflow durability (D0+Queues+DLQ parity) :a1, 2026-03-16, 12d
  Console auth enforced (Access JWT validation) :a2, 2026-03-16, 7d
  OpenAPI contract parity gates :a3, 2026-03-20, 10d

  section Workflow-to-evidence loop
  Step evidence emission required :b1, 2026-03-26, 10d
  Real adapter invocation via dispatch namespace :b2, 2026-03-30, 12d

  section Governance and secrets
  OPA bundles + decision logs integrated :c1, 2026-04-07, 12d
  Vault + GitHub OIDC deploy flow :c2, 2026-04-10, 8d

  section Operational readiness
  SLOs (Logs+Analytics Engine) + runbooks :d1, 2026-04-18, 8d
  Design-partner pilot cut (3-5 tenants) :d2, 2026-04-28, 7d
```

Milestones with owners and acceptance criteria (concise): | Milestone | Owner | Effort | Acceptance criteria | |---|---|---|---| | Workflow durability parity | Platform Eng | L | Workflow run state persists across retries and restarts; step retries bounded; failures route to DLQ using Queues `dead_letter_queue`; scheduled wake-ups via DO alarms. ¹⁹ | | Console auth enforced | Security + Frontend | M | All console/API requests must validate Cloudflare Access JWT (`Cf-Access-Jwt-Assertion`) and enforce `aud` / issuer; no bypass in production config. ⁷ | | OpenAPI contract parity | DevEx | M | CI fails if any documented endpoint is missing/renamed without version bump; golden-path request tests exist per group; response schemas validated (tooling choice unspecified). | | Evidence required per step | Platform Eng | M | Every step writes an evidence envelope to R2 and indexes metadata in D1; evidence objects are content-addressed and immutable by convention; duplicate writes are idempotent (hash-based). ²⁰ | | Adapter invocation real | Integrations | L | Workflow steps call adapters through dispatch namespace (`env.NAMESPACE.get()`); adapters expose capability/introspection endpoint; failures retry and DLQ. ¹⁶ | | OPA governance integrated | Security + Platform Eng | M/L | Policy bundles built and served; OPA decision logs emitted (input, bundle metadata, decision_id) and stored as evidence; default-deny patterns used. ²¹ | | Vault + GitHub OIDC | Security + DevEx | M | CI authenticates to Vault via JWT/OIDC discovery URL `https://token.actions.githubusercontent.com`; long-lived deploy secrets eliminated

(bootstrap exceptions unspecified). ¹⁸ | | SLOs + runbooks | SRE/Platform | M | Workers Logs sampling configured; per-tenant SLIs reported to Analytics Engine; alert thresholds and runbooks exist (alerting tool unspecified). ²² |

Prioritized risk and gap analysis

Scope note: “Repo evidence” below is sourced from the author’s internal repo assessment (provided in conversation). Independent verification is **unspecified** because repository access is not available in this environment.

Priority	Area	Repo evidence (as reported)	Required fix	Owner	Acceptance criteria
P0	Workflow durability	JML engine behaves as a shim; missing persistent orchestration + DLQ semantics	Implement DO-backed workflow run state; step enqueue; retries + DLQ routing via Queues; scheduled wake-ups via DO alarms	Platform Eng	DO alarm-driven resumption works; step execution is queue-based; failed messages land in DLQ after <code>max_retries</code> . ²³
P0	Console/API auth	Console has an auth bypass flag	Enforce Cloudflare Access JWT validation on every request; verify issuer/audience; log auth decisions	Security + Frontend	Requests without valid <code>Cf-Access-Jwt-Assertion</code> are rejected; keys fetched from Access certs endpoint; aud validated. ⁷
P0	Evidence immutability	Evidence layer exists; must be first-class everywhere	Make evidence emission mandatory for workflow steps + policy evaluations; store artifacts in R2; index in D1; adopt content-address keys	Platform Eng	Every run step produces evidence URI; evidence is content-addressed; replays are idempotent. ²⁰

Priority	Area	Repo evidence (as reported)	Required fix	Owner	Acceptance criteria
P1	API contract drift	OpenAPI verify script is lightweight; route prefixes diverge	Add contract tests that validate handler ↔ OpenAPI parity; enforce versioning policy	DevEx	CI fails on undocumented route changes; golden tests exist per endpoint group (framework unspecified).
P1	Governance externalization	OPA/Rego guardrails partially designed but not enforced	Implement OPA bundle distribution and evaluation; record decision logs as evidence	Security + Platform Eng	Bundles built with <code>opa build</code> ; verified/signed; decision logs shipped and stored. ¹⁷
P1	Secrets lifecycle	Secrets handled via worker/GitHub secrets; Vault model not end-to-end	Use Vault JWT auth with GitHub OIDC; retrieve secrets at runtime/ deploy	Security + DevEx	Vault JWT auth configured via OIDC discovery; workflows obtain short-lived Vault token and read only needed paths. ¹⁸
P2	Connector reliability	Adapter generator exists; conformance coverage minimal	Define adapter contract: idempotency, rate limits, error schema, capability discovery	Integrations	Simulator validates core lifecycle endpoints + evidence hooks; adapters expose capability manifest (format unspecified).
P2	Observability and SLO	Observability exists but SLO gates not productized	Configure Workers Logs sampling; emit Analytics Engine datapoints for workflows/ evidence	SRE/ Platform	Per-tenant SLIs queryable via AE SQL API; log sampling configured to manage cost. ²⁴

Priority	Area	Repo evidence (as reported)	Required fix	Owner	Acceptance criteria
P2	Migration readiness	AWS migration planned post-MVP	Implement platform interfaces early; avoid Cloudflare-specific leakage beyond adapters	Platform Eng	Non-platform business logic depends only on interfaces; swapping store/queue backends is bounded. ²⁵

AI governance and evidence model

AI governance model

AI is valuable only when bounded by deterministic policy and evidentiary logging. The recommended control loop:

```

flowchart TB
    S[Signals: HRIS/IdP/App telemetry] --> A[AI Analyzer: propose mapping/remediation]
    A --> G[Policy Gate: OPA evaluate]
    G -->|allow| W[Workflow Step Execution]
    G -->|deny| R[Human review / ticket]
    W --> E[Evidence Writer: envelope + artifacts]
    E --> L[Ledger: R2 objects + D1 index]

```

Policy gating via OPA bundles: AtlasIT should distribute governance as signed bundles (built by `opa build`) so policy can change without redeploying workers; bundles support tar.gz format, manifests, and signing/verification. ²⁶

Decision logging: OPA can emit decision logs containing the input, queried policy, bundle metadata, and a decision ID—supporting audit and offline debugging. ²⁷

Default-deny posture: Rego supports `default allow := false` patterns for deny-by-default access control. ²⁸

Evidence-envelope JSON schema (minimal, canonical, content-addressed)

```

{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://atlasit.example/schemas/evidence-envelope.schema.json",
  "title": "AtlasIT Evidence Envelope",
  "type": "object",
  "required": ["tenantId", "workflowRunId", "stepId", "actor", "eventType",
    "createdAt", "hash"],

```

```

"properties": {
  "tenantId": { "type": "string" },
  "workflowRunId": { "type": "string" },
  "stepId": { "type": "string" },
  "actor": { "type": "string", "description": "atlas.workflow-engine |
adapter.<name> | opa" },
  "eventType": { "type": "string", "description": "step.started|
step.completed|policy.decision|ai.proposal" },
  "subject": {
    "type": "object",
    "properties": {
      "subjectId": { "type": "string" },
      "subjectType": { "type": "string", "enum": ["user", "device",
"service"] }
    },
    "additionalProperties": true
  },
  "artifacts": {
    "type": "array",
    "items": {
      "type": "object",
      "required": ["kind", "uri", "sha256"],
      "properties": {
        "kind": { "type": "string" },
        "uri": { "type": "string" },
        "sha256": { "type": "string", "pattern": "^[a-f0-9]{64}$" }
      },
      "additionalProperties": false
    }
  },
  "policy": {
    "type": "object",
    "properties": {
      "bundleRevision": { "type": "string" },
      "decisionId": { "type": "string" },
      "query": { "type": "string" }
    },
    "additionalProperties": true
  },
  "createdAt": { "type": "string", "format": "date-time" },
  "hash": { "type": "string", "pattern": "^[a-f0-9]{64}$", "description":
"sha256(canonical_json(envelope_without_hash))" }
},
"additionalProperties": true
}

```


Storage and immutability notes (Cloudflare-first, AWS-ready): - Store evidence artifacts in **R2** with Worker bindings (e.g., `r2://evidence/<tenant>/<run>/<step>/<hash>.json`). ¹⁴

- On AWS, store evidence artifacts in **S3 with Object Lock** (Compliance mode where required) to prevent deletion/overwrite during retention windows. ¹⁵

Decision logging example (OPA + AI + workflow step)

OPA decision log fields: OPA decision logs contain events describing policy queries, including input, policy queried, and bundle metadata; OPA returns a `decision_id` for auditing. ²⁷

```
{
  "tenantId": "t_123",
  "workflowRunId": "wr_456",
  "stepId": "assign_group",
  "actor": "opa",
  "eventType": "policy.decision",
  "policy": {
    "query": "data.atlasit.authz.allow",
    "bundleRevision": "7864d60d...",
    "decisionId": "9f2c2c2e-..."
  },
  "input": {
    "action": "group.assign",
    "subject": { "subjectId": "u_789", "roles": ["automation:execute"] },
    "target": { "group": "finance-admin" }
  },
  "result": { "allow": false, "reason": "SoD violation" },
  "createdAt": "2026-03-14T20:30:00Z",
  "hash": "e3b0c44298fc1c149afb4c8996fb924..."
}
```

Minimal JML workflow templates (YAML skeletons)

These templates are intentionally minimal: they define triggers, policy gates, step types, and evidence requirements.

Joiner

```
id: jml.joiner.v1
trigger: { source: hris, event: employee.created }
policy_gates: [tenant_active, subject_unique, no_sod_violation]
steps:
  - { id: create_identity, type: action, ref: idp.create_user, retry: { max:
6 } }
  - { id: assign_access, type: action, ref: idp.assign_groups, retry: { max:
```

```

6 } }
- { id: enforce_mfa, type: remediate, ref: idp.enforce_mfa, retry: { max:
4 } }
- { id: emit_evidence, type: evidence, control_refs: [ACCESS_MFA_ENFORCED] }
on_failure: { dlq: jml.joiner.dlq, strategy: partial_recovery }

```

Mover

```

id: jml.mover.v1
trigger: { source: hris, event: employee.updated }
policy_gates: [subject_exists, mover_change_allowed, no_sod_violation]
steps:
- { id: compute_delta, type: action, ref: atlas.compute_access_delta }
- { id: review_if_risky, type: approval, when: "blast_radius>0.4 or
confidence<0.9" }
- { id: apply_delta, type: action, ref: atlas.apply_access_delta, retry: {
max: 6 } }
- { id: emit_evidence, type: evidence, control_refs:
[LEAST_PRIVILEGE_ENFORCED] }
on_failure: { dlq: jml.mover.dlq, strategy: stop_and_reconcile }

```

Leaver

```

id: jml.leaver.v1
trigger: { source: hris, event: employee.terminated }
policy_gates: [subject_exists, no_hold, privileged_review_if_needed]
steps:
- { id: disable_identity, type: action, ref: idp.disable_user, retry: { max:
6 } }
- { id: revoke_sessions, type: action, ref: idp.revoke_sessions, retry: {
max: 4 } }
- { id: revoke_apps, type: action, ref: atlas.revoke_all_app_access, retry: {
max: 6 } }
- { id: emit_evidence, type: evidence, control_refs:
[OFFBOARDING_DEPROVISION_24H] }
on_failure: { dlq: jml.leaver.dlq, strategy: escalate_immediately }

```

Security and CI/CD essentials for the brief

- **Console/auth hardening:** Validate Cloudflare Access JWT from the `Cf-Access-Jwt-Assertion` header (preferred over cookies), verifying signature against rotating certs and validating `aud`. ⁷

- **Secrets governance:** Use **Vault JWT/OIDC auth** for CI/CD so GitHub Actions can obtain short-lived Vault tokens, with `oidc_discovery_url` and `bound_issuer` set to `https://token.actions.githubusercontent.com`. ¹⁸
- **Workflow durability (Cloudflare):** Use Durable Objects alarms for scheduled resumption and Queues DLQ for terminal failures. ²⁹
- **Workflow durability (AWS future):** Use Step Functions Standard Workflows for long-running, durable orchestration with execution history retained (durability and auditability characteristics vary by workflow type). ¹⁰

Entity references: Cloudflare ³⁰ Amazon Web Services ³¹ Open Policy Agent ³² HashiCorp ³³ GitHub ³⁴

¹ ² ¹² Getting started · Cloudflare D1 docs

https://developers.cloudflare.com/d1/get-started/?utm_source=chatgpt.com

³ ¹⁴ ²⁰ ³² Use R2 from Workers · Cloudflare R2 docs

https://developers.cloudflare.com/r2/api/workers/workers-api-usage/?utm_source=chatgpt.com

⁴ ²³ ²⁹ ³¹ Alarms · Cloudflare Durable Objects docs

https://developers.cloudflare.com/durable-objects/api/alarms/?utm_source=chatgpt.com

⁵ ¹⁹ Cloudflare Queues - Configuration · Cloudflare Queues docs

https://developers.cloudflare.com/queues/configuration/configure-queues/?utm_source=chatgpt.com

⁶ ¹⁶ How Workers for Platforms works · Cloudflare for Platforms docs

https://developers.cloudflare.com/cloudflare-for-platforms/workers-for-platforms/how-workers-for-platforms-works/?utm_source=chatgpt.com

⁷ ³⁰ ³³ Validate JWTs · Cloudflare One docs

https://developers.cloudflare.com/cloudflare-one/access-controls/applications/http-apps/authorization-cookie/validating-json/?utm_source=chatgpt.com

⁸ ²² ²⁴ Workers Logs · Cloudflare Workers docs

https://developers.cloudflare.com/workers/observability/logs/workers-logs/?utm_source=chatgpt.com

⁹ Get started with Workers Analytics Engine · Cloudflare Analytics docs

https://developers.cloudflare.com/analytics/analytics-engine/get-started/?utm_source=chatgpt.com

¹⁰ ²⁵ Choosing workflow type in Step Functions - AWS Step Functions

https://docs.aws.amazon.com/step-functions/latest/dg/choosing-workflow-type.html?utm_source=chatgpt.com

¹¹ Dead Letter Queues · Cloudflare Queues docs

https://developers.cloudflare.com/queues/configuration/dead-letter-queues/?utm_source=chatgpt.com

¹³ ³⁴ Best practices for designing and using partition keys effectively in DynamoDB - Amazon DynamoDB

https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/bp-partition-key-design.html?utm_source=chatgpt.com

¹⁵ Locking objects with Object Lock - Amazon Simple Storage Service

https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock.html?utm_source=chatgpt.com

¹⁷ ²¹ ²⁶ Bundles | Open Policy Agent

https://www.openpolicyagent.org/docs/management-bundles?utm_source=chatgpt.com

18 Use JWT/OIDC authentication | Vault | HashiCorp Developer

https://docs.hashicorp.com/vault/docs/auth/jwt?utm_source=chatgpt.com

27 Decision Logs | Open Policy Agent

https://www.openpolicyagent.org/docs/management-decision-logs?utm_source=chatgpt.com

28 Rego Keyword Examples: default | Open Policy Agent

https://www.openpolicyagent.org/docs/policy-reference/keywords/default?utm_source=chatgpt.com